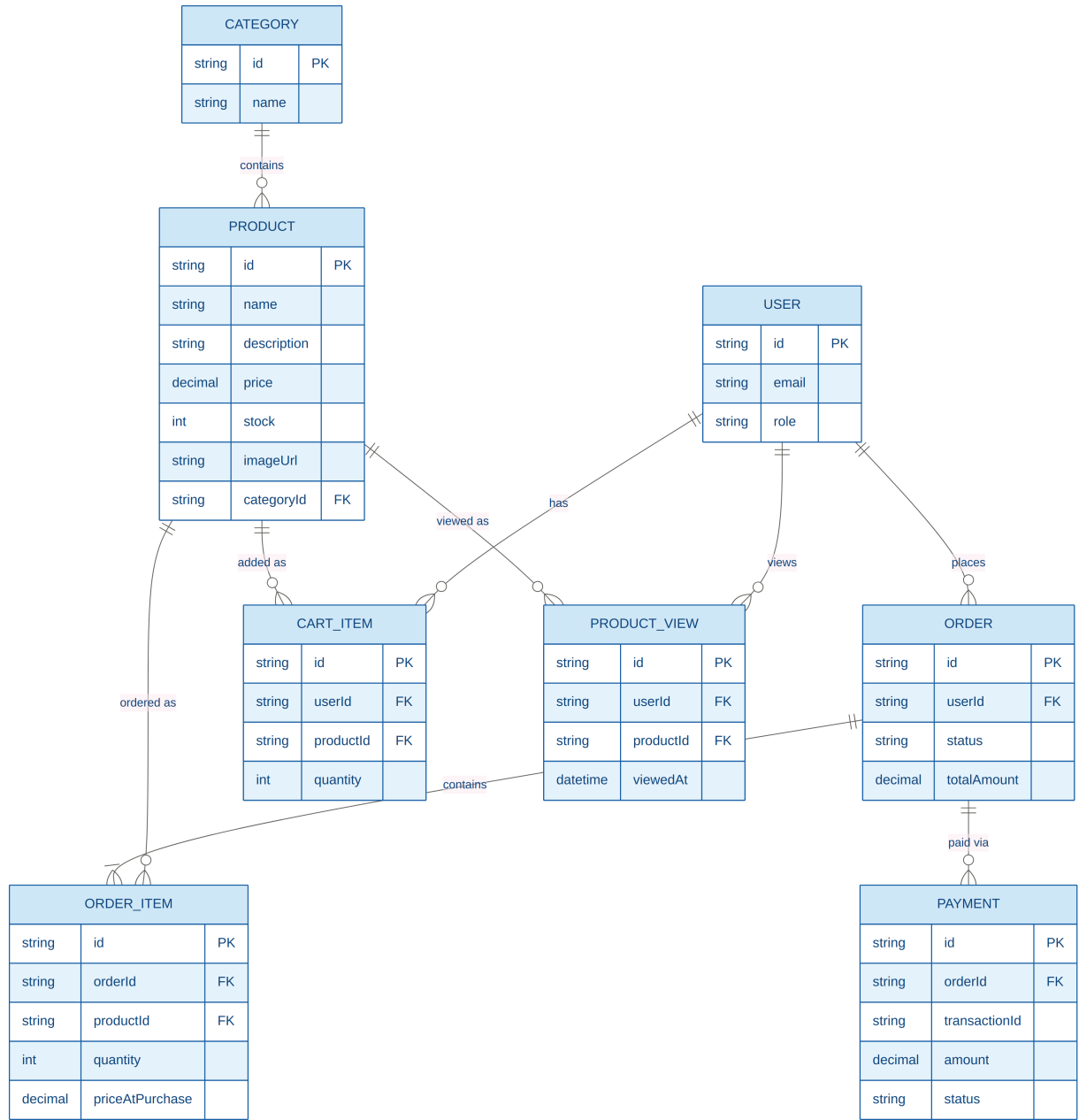


E-Commerce Web App — Entity Relationship Diagram

8-entity schema · PostgreSQL + Prisma ORM



Appendix — Prisma Schema (schema.prisma)

Reference copy of the finalized database schema, generated from the ER diagram on the previous page. All monetary fields use **Decimal** (never Float) to avoid floating-point rounding errors; every child table carries its foreign key as a scalar field paired with a Prisma relation field back to its parent.

```
// ████████████████████████████████████████████████████████████████████████████  
// NOTE: User / Session / Account / Verification models  
// are auto-generated by Better Auth's Prisma adapter.  
// Only the custom `role` field is added by us via  
// Better Auth's config (lib/auth.ts) – shown here  
// as a simplified reference, not hand-written.  
// ████████████████████████████████████████████████████████████████████████████  
  
model User {  
  id          String    @id @default(cuid())  
  email       String    @unique  
  role        Role      @default(USER)  
  
  orders      Order[]  
  cartItems   Cart_Item[]  
  productViews Product_View[]  
  
  createdAt DateTime @default(now())  
  updatedAt DateTime @updatedAt  
}  
  
enum Role {  
  USER  
  ADMIN  
}  
  
model Category {  
  id          String    @id @default(cuid())  
  name        String    @unique  
  products    Product[]  
  
  createdAt DateTime @default(now())  
  updatedAt DateTime @updatedAt  
}  
  
model Product {  
  id          String    @id @default(cuid())  
  name         String  
  description  String  
  price        Decimal  
  stock        Int  
  imageUrl     String  
  
  categoryId  String  
  category    Category @relation(fields: [categoryId], references: [id])  
  
  cartItems   Cart_Item[]  
  orderItems  Order_Item[]  
  productViews Product_View[]  
  
  createdAt   DateTime @default(now())  
  updatedAt   DateTime @updatedAt  
}  
  
model Cart_Item {  
  id          String    @id @default(cuid())  
  
  userId      String  
  user        User      @relation(fields: [userId], references: [id])
```

```

    productId String
    product    Product @relation(fields: [productId], references: [id])

    quantity Int

    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt

    @@unique([userId, productId])
}

enum OrderStatus {
    PENDING
    PAID
    SHIPPED
    DELIVERED
    FAILED
    CANCELLED
}

model Order {
    id          String      @id @default(cuid())

    userId      String
    user        User        @relation(fields: [userId], references: [id])

    status      OrderStatus @default(PENDING)
    totalAmount Decimal

    orderItems  Order_Item[]
    payments    Payment[]

    createdAt   DateTime     @default(now())
    updatedAt   DateTime     @updatedAt
}

model Order_Item {
    id          String      @id @default(cuid())

    orderId      String
    order        Order      @relation(fields: [orderId], references: [id])

    productId    String
    product      Product    @relation(fields: [productId], references: [id])

    quantity     Int
    priceAtPurchase Decimal

    createdAt    DateTime @default(now())
    updatedAt    DateTime @updatedAt
}

enum PaymentStatus {
    PENDING
    SUCCESS
    FAILED
}

model Payment {
    id          String      @id @default(cuid())

    orderId      String
    order        Order      @relation(fields: [orderId], references: [id])

    transactionId String
    amount        Decimal
    status        PaymentStatus @default(PENDING)
}

```

```
    createdAt    DateTime    @default(now())
    updatedAt    DateTime    @updatedAt
}
```

```
model Product_View {
  id            String      @id @default(cuid())

  userId        String
  user          User        @relation(fields: [userId], references: [id])

  productId     String
  product       Product     @relation(fields: [productId], references: [id])

  viewedAt     DateTime @default(now())
}
```